Lab 09: Extracting Data from PDFs

INFO 4617/5617 · Week 9 · Chapter 9, Extracting Data from PDFs

Today we code together. Each step below says what we're building and why, and its code cells hold only the comments that frame them: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll take the City of Boulder's PDFs apart the way the chapter does: see what a page actually stores, sort pages into born-digital, scanned, and scanned with a text layer, pull text with pypdf and tables with pdfplumber, read a picture of a table with OCR, and check every extraction. We end with eight years of the city's sales and use tax, read from seven year-end revenue reports.

The files: the chapter's copies of the city's records, kept in the book's repository, so that a whole class doesn't hit the city's server at once. Only one step (Step 3) asks the city's portal itself, and in class only the instructor runs it.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Step 0 imports pypdf, pdfplumber, and pytesseract; if one stops with `ModuleNotFoundError`, run `%conda install -y -c conda-forge pypdf pdfplumber pytesseract tesseract` once in a new cell, as the chapter's "Installing pypdf, pdfplumber, and pytesseract" says, then restart the kernel.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in and was run on 2026-10-21: the book's saved copies from GitHub, and, in Step 3, two live requests to the city's records portal. The PDFs don't change, so a rerun gives the same figures, except where library versions differ, and except Step 7's microsprint timings, which depend on the machine. This run used pypdf 6.21.2, pdfplumber 0.11.10, and Tesseract 5.3.4; the chapter's outputs came from pypdf 6.20.1 and Tesseract 5.5.3. Three outputs differ from the chapter's. In Step 7, Tesseract parsed 29 of Exhibit 3's 30 rows and set 1 aside, where the chapter reports 28 and 2. In Step 4, pypdf's layout mode keeps every word whole ("Council Chambers", "Carried"), where the chapter prints "C ham bers" and "Car r ied"; and in Step 2, page 3 of the December 2024 report has 2,781 characters, where the chapter prints 2,782. Those two come out the same with pypdf 6.20.1, so they aren't a version difference. Step 3's folder listing will change if the city adds a report. The lab follows the textbook at commit d9306b7, and writes the chapter's functions, except `get_saved_copy()`, as straight lines. Every cell was run.

Step 0 — Set up, and get the first file (5 minutes)

Import what reads PDFs, and print the libraries' versions, and Tesseract's: `pytesseract` runs a separate program, Tesseract, and a missing program shows up here as `TesseractNotFoundError`. Then define the chapter's `BOOK_DATA`, the book's folder of saved files on GitHub, `https://raw.githubusercontent.com/cuinfoscience/We Data-Science-Book/main/data/ch-09/`; `HEADERS`; and a `pdfs` folder, and download the December 2024 revenue report into it. A PDF is bytes, not text, so save `response.content`, not `response.text`. Later steps import what they use when they first use it, as the chapter does. The chapter's "This Chapter's Files" has the code.

```
# Import requests, Path (from pathlib), pandas as pd, pypdf, pdfplumber, pytesseract, and
 ↪  PdfReader (from pypdf)
# Print the versions of pypdf, pdfplumber, and pytesseract, and Tesseract's, from
 ↪  pytesseract.get_tesseract_version()
import requests
from pathlib import Path
import pandas as pd
import pypdf
import pdfplumber
```

```
import pytesseract
from pypdf import PdfReader

print(f"pypdf {pypdf.__version__} | pdfplumber {pdfplumber.__version__} | pytesseract
 ↪  {pytesseract.__version__}")
print(f"Tesseract {pytesseract.get_tesseract_version()}")
```

```
pypdf 6.21.2 | pdfplumber 0.11.10 | pytesseract 0.3.13
Tesseract 5.3.4
```

```
# Define BOOK_DATA (the URL above), HEADERS (a User-Agent with your email, as in week 2),
#   and PDF_DIR = Path("pdfs"); make the folder
BOOK_DATA =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-09/"
HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
PDF_DIR = Path("pdfs")
PDF_DIR.mkdir(exist_ok=True)
```

```
# Request BOOK_DATA + "revenue-report-2024-12.pdf" with HEADERS; save response.content to PDF_DIR
 ↪  as report_2024; print its size
response = requests.get(BOOK_DATA + "revenue-report-2024-12.pdf", headers=HEADERS)
report_2024 = PDF_DIR / "revenue-report-2024-12.pdf"
report_2024.write_bytes(response.content)  # a PDF is bytes, so save the bytes as they came
print(f"{report_2024}: {report_2024.stat().st_size:,} bytes")
```

```
pdfs/revenue-report-2024-12.pdf: 723,920 bytes
```

💬 Discuss

- In week 2 we put contact information in the User-Agent. Who reads this one, now that the request goes to GitHub rather than to the city?
- Our cell saves whatever comes back. If GitHub answered 404 or 429, what would `revenue-report-2024-12.pdf` hold, and what would the chapter's `raise_for_status()` have done instead?
- Why does the book serve its own copies instead of sending a class of forty to the city's records portal?

Step 1 — What a PDF page draws (5 minutes)

A PDF page is a list of drawing instructions: characters at coordinates, lines and rectangles, and images. Nothing in it says "word", "line", or "table"; every tool infers those from position. pdfplumber lists every object on a page with its position in points (1/72 inch) from the left edge (`x0`) and the top (`top`). Look at page 2, where the report's tables start. Then read what the file says about itself with pypdf's `metadata`, and whether it is tagged, with a structure tree for screen readers. The chapter's "What's Inside a PDF" has more, including the drawing instructions themselves.

```
# Open report_2024 with pdfplumber; for page 2, print the page count and the page's width and
 ↪  height,
#   seven of page.chars (text, x0, top, fontname, size), and how many chars, lines, rects, and
 ↪  images it draws
with pdfplumber.open(report_2024) as pdf:
    page = pdf.pages[1]  # the second page
    print(f"{len(pdf.pages)} pages; this one is {page.width} by {page.height} points")
    for char in page.chars[1:8]:
        print(f"{char['text']!r} {char['x0']:.1f} {char['top']:.1f} {char['fontname']}
         ↪  {char['size']:.1f}")
    print(f"{len(page.chars)} characters, {len(page.lines)} lines, {len(page.rects)} rectangles,
     ↪  {len(page.images)} images")
```

```
15 pages; this one is 612.0 by 792.0 points
'C' 54.0 30.4 CALYKT+Arial-BoldMT 11.0
'i' 61.9 30.4 CALYKT+Arial-BoldMT 11.0
't' 65.0 30.4 CALYKT+Arial-BoldMT 11.0
'y' 68.8 30.4 CALYKT+Arial-BoldMT 11.0
' ' 74.9 30.4 CALYKT+Arial-BoldMT 11.0
'o' 78.0 30.4 CALYKT+Arial-BoldMT 11.0
'f' 84.6 30.4 CALYKT+Arial-BoldMT 11.0
2222 characters, 0 lines, 549 rectangles, 0 images
```

```
# Open report_2024 with PdfReader as reader; print the metadata's title, subject, creator, and
 ↪  creation_date,
#   and whether reader.trailer["/Root"] holds "/StructTreeRoot" (a tagged PDF)
reader = PdfReader(report_2024)
meta = reader.metadata
print(f"{meta.title} | {meta.subject} | {meta.creator} | {meta.creation_date}")
print(f"tagged: {'/StructTreeRoot' in reader.trailer['/Root']}")
```

```
REVENUE REPORT | March 2018 | Acrobat PDFMaker 25 for Word | 2025-05-01 13:20:25-06:00
tagged: True
```

💬 Discuss

- The first characters spell ”City of” one at a time, and the table's rules are hundreds of thin rectangles. What does a table finder have to rebuild before it can find a single cell?
- The subject of a report on December 2024 says March 2018. What else in a file's metadata would you hesitate to rely on, and what is the creator still good for?
- Week 4's XML and JSON store their structure in the file. What does a PDF store instead, and what does that cost the person who wants its data?

Step 2 — What kind of PDF is it? (9 minutes)

Every step from here on needs another of the book's files, so first gather Step 0's download into the chapter's `get_saved_copy(name)`: it downloads a file only if it isn't in `pdfs` yet, and pauses a second after each download, so that a loop of them is polite.

Then sort the pages, because the right tool depends on how each was made. **Born-digital** pages have characters to extract; **scanned** pages are one image and no characters, so they need OCR; **scanned with a text layer** pages are one image plus text that someone's OCR already produced, of unknown quality. For three files, count each page's characters of `extract_text()` and its `images`. Then ask the 2018 minutes who made them, and read their first lines. The chapter's ”What Kind of PDF Is It?” has the routing table; it wraps the count in a function, `page_profile()`.

```
# Import time. Define get_saved_copy(name), Step 0's lines gathered: path = PDF_DIR / name; if
 ↪  the path doesn't exist,
#   request BOOK_DATA + name, save the bytes, and sleep a second; return the path
import time

def get_saved_copy(name):
    path = PDF_DIR / name
    if not path.exists():  # download only the first time
        response = requests.get(BOOK_DATA + name, headers=HEADERS)
        path.write_bytes(response.content)
        time.sleep(1)  # a pause after each download, so a loop of them is polite
    return path
```

```
# For each of the three files, collect every page's len(page.extract_text()) in characters and
 ↪  len(page.images) in images;
```

```
#   print the file's name and both lists
for name in ["revenue-report-2024-12.pdf", "revenue-report-2018.pdf",
↪  "council-minutes-2018-12-04.pdf"]:
    characters = []
    images = []
    for page in PdfReader(get_saved_copy(name)).pages:
        characters.append(len(page.extract_text()))
        images.append(len(page.images))
    print(f"{name}\n  characters: {characters}\n  images:     {images}")
```

```
revenue-report-2024-12.pdf
  characters: [1332, 2279, 2781, 648, 738, 393, 491, 1356, 1390, 676, 3429, 1519, 3797, 4624,
  ↪  265]
  images:     [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

revenue-report-2018.pdf
  characters: [1491, 1919, 143, 474, 42, 191, 374, 269, 907, 1273, 3048, 1334, 3031, 3964, 5569]
  images:     [1, 0, 2, 1, 1, 1, 3, 3, 2, 1, 0, 0, 0, 0, 0]

council-minutes-2018-12-04.pdf
  characters: [1718, 1869, 1729, 2381, 2776, 1706, 1665, 1673, 320]
  images:     [1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
# Open council-minutes-2018-12-04.pdf as minutes; print its creator and creation_date, and the
↪  first three lines of page 1's text
minutes = PdfReader(get_saved_copy("council-minutes-2018-12-04.pdf"))
print(f"{minutes.metadata.creator} | {minutes.metadata.creation_date}")
for line in minutes.pages[0].extract_text().splitlines()[:3]:
    print(line.strip())
```

```
Canon  | 2019-01-22 10:08:22-07:00
CITY OF BOULDER
CITY COTJNCIL MEETING
Municipal Building, 1777 Broadway
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 5's quality check.

Do: take one of two files the chapter mentions but doesn't profile, `council-minutes-2000-12-19.pdf` or `revenue-report-2021-12.pdf`, and give each page a route from the chapter's table: born-digital (no image), OCR (an image and under 100 characters), or test the text layer (an image and more text). Print each page's number, characters, and route. **Share:** your file, how many pages take each route, and which pages you'd send to OCR.

```
# Microsprint: for each page of one file (enumerate(..., start=1)), print its number, its
↪  characters, and its route:
#   no image -> born-digital; an image and under 100 characters -> OCR; an image and more text ->
↪  test the text layer
for number, page in enumerate(PdfReader(get_saved_copy("council-minutes-2000-12-19.pdf")).pages,
↪  start=1):
    characters = len(page.extract_text())
    if len(page.images) == 0:
        print(f"page {number}: {characters} characters, born-digital")
    elif characters < 100:
        print(f"page {number}: {characters} characters, OCR")
    else:
        print(f"page {number}: {characters} characters, test the text layer")
```

```
page 1: 2207 characters, test the text layer
page 2: 0 characters, OCR
page 3: 0 characters, OCR
page 4: 0 characters, OCR
page 5: 1693 characters, test the text layer
page 6: 1575 characters, test the text layer
page 7: 0 characters, OCR
page 8: 0 characters, OCR
page 9: 0 characters, OCR
```

💬 **Discuss**

- How many pages of your file go to OCR? Would one character count for the whole file have told you which ones?
- The scanner's own OCR turned ”COUNCIL” into ”COTJNCIL”. Why is a text layer you didn't make something to test rather than something to trust?
- Week 6's exercises cached a page's raw HTML before parsing it, and `get_saved_copy()` does the same. If a download had saved an error page instead of a PDF, what would every rerun read, and which cell would tell you?

Step 3 — Where the files came from: the city's portal (5 minutes)

In class, only the instructor runs this step's three cells. On October 6, 2026, the city's server stopped answering after 13 to 17 requests in a row; a room of us running them at once would look like one client sending forty. Write them now, and run them at home. Nothing later in the lab needs them, so before Friday's **Run All Above**, comment them out (select a cell's lines and press Ctrl+/ or Cmd+/), or a room of us will send them at once after all.

The saved copies came from the city's Laserfiche WebLink portal. Its folder page is a shell that JavaScript fills in, and the browser's Network tab shows where the list comes from: a POST to `GetFolderListing2` that sends a folder's number as JSON and gets JSON back. The city's server leaves out an intermediate certificate, so Python's default check fails with `CERTIFICATE_VERIFY_FAILED`; pass the book's bundle with `verify=bundle`, never `verify=False`. Then download one report, after the chapter's 30-second pause, and look at what came back before trusting it: the content type, the five bytes `%PDF-` that start every PDF, and the file's SHA-256 hash against the book's `manifest.csv`, its record of where each copy came from. Our cells assume each request works; the chapter's add `raise_for_status()` and a `timeout`. The chapter's ”Getting the Files” and ”Appendix: Technical Notes” have the code.

```
# In class, only the instructor runs this step. Import hashlib; get boulder-ca-bundle.pem as
 ↪  bundle and read manifest.csv
#   with get_saved_copy(); define PORTAL and the folder page's JSON body for folder 194345
import hashlib

bundle = get_saved_copy("boulder-ca-bundle.pem")
manifest = pd.read_csv(get_saved_copy("manifest.csv"))
PORTAL = "https://documents.bouldercolorado.gov/WebLink/"
body = {"repoName": "LF8PROD2", "folderId": 194345, "getNewListing": True,
        "start": 0, "end": 100, "sortColumn": "", "sortAscending": True}
```

```
# POST body as JSON to PORTAL + "FolderListingService.aspx/GetFolderListing2" with HEADERS and
 ↪  verify=bundle; folder = its JSON's "data"
#   Print the folder's path, then each entry's entryId, name, and kind (type 0 is a folder; else
 ↪  data[1] is its page count)
response = requests.post(PORTAL + "FolderListingService.aspx/GetFolderListing2", json=body,
 ↪  headers=HEADERS, verify=bundle)
folder = response.json()["data"]
print(f"{folder['path']} | {folder['totalEntries']} entries")
```

```
for entry in folder["results"]:
    if entry["type"] == 0:
        print(f"{entry['entryId']} {entry['name']} (folder)")
    else:
        print(f"{entry['entryId']} {entry['name']} ({entry['data'][1]} pages)")
```

```
\Central Records\Finance\Reports\Revenue Reports | 7 entries
194346 2021 (folder)
194359 2022 (folder)
194372 2023 (folder)
194385 2024 (folder)
194398 Revenue Report 2018 (15 pages)
194399 Revenue Report 2019 (14 pages)
194400 Revenue Report 2020 (16 pages)
```

```
# Sleep 30 seconds; GET ElectronicFile.aspx for docid 194397 with verify=bundle; print its
 ↪  Content-Type, size, and first 5 bytes
#   Find the manifest's row for the docid, and print whether the SHA-256 of what came back
 ↪  matches its sha256
time.sleep(30)  # the chapter's pause between requests to the city's server
docid = 194397  # Revenue Report 2024.12
response = requests.get(PORTAL + f"ElectronicFile.aspx?docid={docid}&dbid=0&repo=LF8PROD2",
 ↪  headers=HEADERS, verify=bundle)
print(f"{response.headers['Content-Type']} | {len(response.content)} bytes |
 ↪  {response.content[:5]}")
record = manifest[manifest["docid"] == docid].iloc[0]
print(f"{record['file']}, retrieved {record['retrieved_utc']}")
print(f"same file as the book's copy: {hashlib.sha256(response.content).hexdigest() ==
 ↪  record['sha256']}")
```

```
application/pdf | 723920 bytes | b'%PDF-'
revenue-report-2024-12.pdf, retrieved 2026-10-06T06:24:30Z
same file as the book's copy: True
```

💬 Discuss

- Week 8 found hidden APIs in the Network tab. Which parts of this request (the folder number, `LF8PROD2`, the body) would break first if the city upgraded its portal?
- Python's default check failed and the book's bundle passed. Why is `verify=False` the wrong fix, even for public records?
- Our cells leave out the chapter's `raise_for_status()` and `timeout=`. If the server stopped answering, as it did after 13 to 17 requests, what would the cell do, and which printed line would catch a 200 with an empty body?

Step 4 — Text two ways: pypdf and pdfplumber (8 minutes)

The minutes of December 5, 2024 were written in Word and saved as PDF, so every page has characters. pypdf writes them out in the order the page draws them; its `extraction_mode="layout"` places each piece by its position instead. pdfplumber starts from where each character ends up, grouping characters into words when the gap is under 3 points (`x_tolerance`) and words into lines when their tops line up. Print the top of page 1 all three ways and compare. The chapter's "Text with pypdf" shows, in the page's drawing instructions, why Word's first letters break off.

```
# Open council-minutes-2024-12-05.pdf with PdfReader as minutes_2024; text = page 1's
 ↪  extract_text(); print its first 13 lines
minutes_2024 = PdfReader(get_saved_copy("council-minutes-2024-12-05.pdf"))
text = minutes_2024.pages[0].extract_text()
```

```
for line in text.splitlines()[:13]:
    print(line.rstrip())
```

```
P age 1
CI
TY COUNCIL MEETING
Council Chambers
Thursday, December 5, 2024
MINUTES
1. C
all to Order and Roll Call:
M
ayor Brockett called the meeting to order at 6:00 p.m.
C
ouncil Members present: Benjamin, Brockett, Folkerts, Marquis, Schuchard,
Speer, Wallach, Winer
```

```
# Print the first lines of page 1's extract_text(extraction_mode="layout"), skipping blank lines
 ↪  and shrinking runs of spaces
layout = minutes_2024.pages[0].extract_text(extraction_mode="layout")
for line in layout.splitlines()[:18]:
    if line.strip():
        print(" ".join(line.split()))
```

```
CITY COUNCIL MEETING
Council Chambers
Thursday, December 5, 2024
MINUTES
1. Call to Order and Roll Call:
Mayor Brockett called the meeting to order at 6:00 p.m.
Council Members present : Benjamin, Brockett, Folkerts, Marquis, Schuchard,
Speer, Wallach, Winer
Virtually present: Adams
Motion Made By/Seconded Vote
Motion to AMEND the agenda to: Wallach / Benjamin Carried 9:0
```

```
# Open the same file with pdfplumber and print the first 11 lines of page 1's extract_text()
with pdfplumber.open(get_saved_copy("council-minutes-2024-12-05.pdf")) as pdf:
    for line in pdf.pages[0].extract_text().splitlines()[:11]:
        print(line)
```

```
CITY COUNCIL MEETING
Council Chambers
Thursday, December 5, 2024
MINUTES
1. Call to Order and Roll Call:
Mayor Brockett called the meeting to order at 6:00 p.m.
Council Members present: Benjamin, Brockett, Folkerts, Marquis, Schuchard,
Speer, Wallach, Winer
Virtually present: Adams
Motion Made By/Seconded Vote
Motion to AMEND the agenda to: Wallach / Benjamin Carried 9:0
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 4, on a table instead of prose.

Do: extract page 2 of `report_2024`, the page with the sales and use tax summary, all three ways, and print, from each, only the lines that hold `"Total Sales"` or `"current economic activity"`. **Share:** whether the three extractions agree on these rows, and what a pattern that reads one row per line would take as the subtotal's label, ”Total Sales & Use Tax based on current economic activity”, which wraps onto two lines on the page.

```
# Microsprint: extract page 2 of report_2024 with pypdf (plain and layout) and pdfplumber;
#   from each, print only the lines that contain "Total Sales" or "current economic activity"
plain = PdfReader(report_2024).pages[1].extract_text()
layout = PdfReader(report_2024).pages[1].extract_text(extraction_mode="layout")
with pdfplumber.open(report_2024) as pdf:
    plumber = pdf.pages[1].extract_text()
for tool, page_text in [("pypdf, plain", plain), ("pypdf, layout", layout), ("pdfplumber",
 ↪  plumber)]:
    for line in page_text.splitlines():
        if "Total Sales" in line or "current economic activity" in line:
            print(f"{tool}: {' '.join(line.split())}")
```

```
pypdf, plain: Total Sales & Use Tax based on
pypdf, plain: current economic activity $172,401,668 $171,006,199 -$1,395,469 -0.81% -0.81%
pypdf, plain: Total Sales and Use Tax $175,517,129 $173,908,431 -$1,608,698 -0.92% -0.92%
pypdf, layout: Total Sales & Use Tax based on
pypdf, layout: current economic activity $172,401,668 $171,006,199 -$1,395,469 -0.81% -0.81%
pypdf, layout: Total Sales and Use Tax $175,517,129 $173,908,431 -$1,608,698 -0.92% -0.92%
pdfplumber: Total Sales & Use Tax based on
pdfplumber: current economic activity $172,401,668 $171,006,199 -$1,395,469 -0.81% -0.81%
pdfplumber: Total Sales and Use Tax $175,517,129 $173,908,431 -$1,608,698 -0.92% -0.92%
```

💬 **Discuss**

- pypdf's plain mode broke lines. The chapter says layout mode breaks words instead (”C ham bers”), but ours didn't. What would you check before relying on either mode's output for a new file?
- In the microsprint all three tools agreed, and all three put the subtotal's figures on its label's second line. What would a one-row-per-line pattern call that row, and which check would notice?
- pypdf printed ”P age 1” first, though the page number sits at the bottom. What does that tell you about the order a PDF is drawn in?

Step 5 — Repair the text, then find things in it (8 minutes)

Repair what pypdf broke with regular expressions, matching what the extraction produced rather than what the page shows: a pattern for ”Page 1” would miss ”P age 1”. Join the text of all eight pages, then make the chapter's four repairs, one `re.sub()` each: drop the page number; rejoin a first letter that Word drew apart (”M” and ”ayor”); collapse runs of spaces and tabs but **not** newlines, because a heading is recognizable by sitting on its own line; and strip each line and drop the empty ones. Then run the chapter's five patterns with `re.findall()`. The chapter's ”Cleanup Strategies” gathers the repairs into a function, `clean_page_text()`, and ”Regular Expressions for PDF Text” explains each pattern.

```
# Import re. Collect the extract_text() of every page of minutes_2024 in a list, and join them
 ↪  with "\n" as raw_text
import re

page_texts = []
for page in minutes_2024.pages:
    page_texts.append(page.extract_text())
raw_text = "\n".join(page_texts)
```

```
# full_text: raw_text after four re.sub() repairs: remove ^P ?age \d+ lines (re.MULTILINE); join
 ↪  \b([A-Z]{1,2})\n before a letter;
#   replace [ \t]+ with one space; and replace " *\n\s*" with "\n", which strips each line and
 ↪  drops empty ones
# Print the first 8 lines
full_text = re.sub(r"^P ?age \d+[ \t]*$", "", raw_text, flags=re.MULTILINE)  # the page number,
 ↪  "P age 1"
full_text = re.sub(r"\b([A-Z]{1,2})\n(?=[A-Za-z])", r"\1", full_text)        # "M" and "ayor"
 ↪  become "Mayor"
full_text = re.sub(r"[ \t]+", " ", full_text)                                 # spaces and tabs,
 ↪  but not newlines
full_text = re.sub(r" *\n\s*", "\n", full_text).strip()                       # strip each line;
 ↪  drop empty ones
for line in full_text.splitlines()[:8]:
    print(line)
```

```
CITY COUNCIL MEETING
Council Chambers
Thursday, December 5, 2024
MINUTES
1. Call to Order and Roll Call:
Mayor Brockett called the meeting to order at 6:00 p.m.
Council Members present: Benjamin, Brockett, Folkerts, Marquis, Schuchard,
Speer, Wallach, Winer
```

```
# With re.findall() on full_text, print the dates in words (MONTH + r" \d{1,2}, \d{4}") and the
 ↪  dates with slashes,
#   the first and last clock times and how many there are, the (Carried|Failed) n:n tallies, and
 ↪  the dollar amounts
MONTH =
 ↪  r"(?:January|February|March|April|May|June|July|August|September|October|November|December)"
print(re.findall(MONTH + r" \d{1,2}, \d{4}", full_text))
print(re.findall(r"\d{1,2}/\d{1,2}/\d{2,4}", full_text))
times = re.findall(r"\d{1,2}:\d{2} [ap]\.m\.", full_text)
print(f"{times[0]} to {times[-1]} ({len(times)} times in all)")
print(re.findall(r"(Carried|Failed) (\d+):(\d+)", full_text))
print(re.findall(r"\$[\d,]+(?:\.\d{2})?", full_text))
```

```
['December 5, 2024', 'November 5, 2024', 'November 5, 2024']
['12/05/24', '12/19/24']
6:00 p.m. to 10:43 p.m. (12 times in all)
[('Carried', '9', '0'), ('Carried', '9', '0')]
['$20,000,000']
```

⏱️ Microsprint (5 minutes, alone or in pairs)

The chapter's advice, ”check a repair written for one file on the next, and print what it changed”, cut to five minutes.

Do: on every page of `council-minutes-2023-12-07.pdf`, count the places where the second repair's pattern, `\b[A-Z]{1,2}\n(?=[A-Za-z])`, would join two lines. A join is suspicious when its capitals don't start a word, that is, when the character before them is neither a letter nor a space, as in ”A-F”; the pattern `\S*[^\w\s][A-Z]{1,2}\n\S+` finds those. Print each as the word the join would make. (The other joins include the first names of members of the public who spoke, so print counts, not words.) **Share:** how many joins, how many look suspicious, and which of those is really wrong.

```
# Microsprint: for each page of council-minutes-2023-12-07.pdf, add the number of the repair
↪  pattern's matches to joins;
#   print each suspicious join with its page, with the "\n" removed. Print joins
minutes_2023 = PdfReader(get_saved_copy("council-minutes-2023-12-07.pdf"))
joins = 0
for number, page in enumerate(minutes_2023.pages, start=1):
    page_text = page.extract_text()
    joins += len(re.findall(r"\b[A-Z]{1,2}\n(?=[A-Za-z])", page_text))
    for match in re.findall(r"\S*[^\w\s][A-Z]{1,2}\n\S+", page_text):
        word = match.replace("\n", "")
        print(f"page {number}: {word}")
print(f"{joins} joins in all")
```

```
page 1: In-Person
page 3: A-FWallach
page 3: In-Person:
30 joins in all
```

💬 Discuss

- The join repair was right every time on the December 5 minutes and wrong at least once here. When has a repair been tested enough to run on a hundred files?
- The tally pattern found two votes, both in the motion tables. How would you find out how many votes it missed, such as ”by a vote of 9:0”?
- Week 7 dropped stopwords before counting words. What else would a count of a topic word like ”housing” need before you compared one meeting with another?

Step 6 — Tables with pdfplumber (13 minutes)

pdfplumber's table finder rebuilds a grid from the page's rules and the sides of its rectangles (the lines strategy), and makes a cell wherever rules close a box. Run `find_tables()` on the summary page, and check its first table against the page with `search()`: the totals row has no vertical rules beside it, so it falls outside the table, and nothing raises an error. Then fix it as the chapter does: give the columns yourself as `explicit_vertical_lines` (the left and right edges of the cells it did find), keep the page's horizontal rules, and crop just below the totals row. Last, turn the rows into a DataFrame: take the label and the two years by position, keep the rows with every cell filled, and turn the figures into numbers with `pd.to_numeric()` once the dollar signs and commas are gone, as week 6 did with a scraped table. The chapter's ”Finding Tables” lists every table setting, and gathers the last part into a function, `summary_frame()`.

```
# Open report_2024 with pdfplumber as report; summary_page = page 2; tables = its find_tables()
#   For each table, print its bbox (left, top, right, bottom), rounded, and its number of rows
report = pdfplumber.open(report_2024)  # closed at the end of this step
summary_page = report.pages[1]
tables = summary_page.find_tables()
for table in tables:
    left, top, right, bottom = table.bbox
    print(f"left {left:.0f}, top {top:.0f}, right {right:.0f}, bottom {bottom:.0f}:
    ↪  {len(table.rows)} rows")
```

```
left 54, top 188, right 588, bottom 375: 12 rows
left 60, top 203, right 232, bottom 231: 2 rows
left 54, top 449, right 577, bottom 632: 13 rows
left 459, top 470, right 505, bottom 495: 2 rows
```

```
# default_rows = the first table's extract(); print the first six cells of its last row
#   Find the "Total Sales and Use Tax" row with search(); print the table's bottom and the row's
↪  top and bottom
```

```
default_rows = tables[0].extract()
print(default_rows[-1][:6])
total_row = summary_page.search("Total Sales and Use Tax")[0]
print(f"the table ends at {tables[0].bbox[3]:.0f}; the totals row runs from
 ↪  {total_row['top']:.0f} to {total_row['bottom']:.0f}")
```

```
['Use Tax Audit Revenue', None, '1,498,943', None, '643,062', '-855,881']
the table ends at 375; the totals row runs from 380 to 391
```

```
# Collect the left and right edge of every cell of the first table (cell[0] and cell[2], rounded
 ↪  to 2 places) in a set; sort it
first = tables[0]
edges = set()
for cell in first.cells:
    edges.add(round(cell[0], 2))  # the cell's left edge
    edges.add(round(cell[2], 2))  # and its right edge
edges = sorted(edges)
```

```
# The fix: settings with "explicit" vertical lines at edges and the "lines" strategy for rows;
 ↪  crop the table's area at 395 points
#   fixed_rows = the area's extract_table(settings); close report; print the rows before and
 ↪  after, and the last row's first six cells
settings = {"vertical_strategy": "explicit", "explicit_vertical_lines": edges,
 ↪  "horizontal_strategy": "lines"}
area = summary_page.crop((first.bbox[0], first.bbox[1], first.bbox[2], 395))  # below the totals
 ↪  row, above the footnote
fixed_rows = area.extract_table(settings)
report.close()
print(f"{len(default_rows)} rows before, {len(fixed_rows)} after")
print(fixed_rows[-1][:6])
```

```
12 rows before, 13 after
['', 'Total Sales and Use Tax', '', '$175,517,129', '$173,908,431', '-$1,608,698']
```

```
# summary_df: columns 1, 3, and 4 of fixed_rows, named source, ytd_2023, and ytd_2024; blank
 ↪  cells to pd.NA, then dropna()
#   Put each label on one line; turn both years into numbers with pd.to_numeric() after removing
 ↪  "$" and ","; print it
summary_df = pd.DataFrame(fixed_rows).iloc[:, [1, 3, 4]]
summary_df.columns = ["source", "ytd_2023", "ytd_2024"]
summary_df = summary_df.replace("", pd.NA).dropna()  # keep the rows with a label and both
 ↪  figures
summary_df["source"] = summary_df["source"].str.replace("\n", " ")
summary_df["ytd_2023"] = pd.to_numeric(summary_df["ytd_2023"].str.replace(r"[$,]", "",
 ↪  regex=True))
summary_df["ytd_2024"] = pd.to_numeric(summary_df["ytd_2024"].str.replace(r"[$,]", "",
 ↪  regex=True))
print(summary_df.to_string(index=False))
```

```
                                          source  ytd_2023  ytd_2024
                                       Sales Tax 137732945 139069840
                             Rec MJ Additional 3.5%*   1386752   1184018
                                Business Use Tax  10292756   9855472
                           Construction Use Tax  16541587  14930902
                            Motor Vehicle Use Tax   6447628   5965968
Total Sales & Use Tax based on current economic activity 172401668 171006199
                             Sales Tax Audit Revenue   1616518   2259170
```

```
                  Use Tax Audit Revenue   1498943    643062
Total Sales and Use Tax 175517129 173908431
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A check of Step 6's finding on another year, after Additional Exercise 2. The chapter shows the totals row falling outside the table in December 2024's report, and says the other reports' tables have different rules.

Do: open one other year-end report (`revenue-report-2018.pdf`, `-2019.pdf`, `-2020.pdf`, `-2021-12.pdf`, `-2022-12.pdf`, or `-2023-12.pdf`); its summary is on page 2, except 2020's, which is on page 3. Run `find_tables()`, and check with `search()` whether the first table reaches the `"Total Sales and Use Tax"` row. **Share:** your year, where the table ends and where the totals row sits, and the label of the table's last row.

```
# Microsprint: open another report's summary page; take the first table from find_tables() and
 ↪  the totals row from search()
#   Print where the table ends and where the totals row runs, and the first cell of the table's
 ↪  last row
with pdfplumber.open(get_saved_copy("revenue-report-2018.pdf")) as pdf:
    page = pdf.pages[1]
    first_2018 = page.find_tables()[0]
    total_2018 = page.search("Total Sales and Use Tax")[0]
    print(f"the table ends at {first_2018.bbox[3]:.0f}; the totals row runs from
     ↪  {total_2018['top']:.0f} to {total_2018['bottom']:.0f}")
    print(first_2018.extract()[-1][0])
```

```
the table ends at 523; the totals row runs from 529 to 540
Business Use Tax Audit Revenue
```

💬 Discuss

- The default finder dropped the totals row and raised nothing. What told us, and what would have told us if we hadn't looked at the page?
- Did the default finder drop the totals row in your year too? What does the class's answer say about checking one file and trusting the rest?
- Week 6's `pd.read_html()` turned an HTML table into a DataFrame in one call. What does HTML's `<td>` give you for free that a PDF table doesn't?

Step 7 — No text layer: OCR (11 minutes)

Optical character recognition turns a picture of text into characters. pytesseract runs Tesseract on a Pillow image, which pypdf hands you as `page.images[0].image`. First, page 7 of the December 2018 minutes, where the scanner's own text layer turned the motion into ”CouNcrr, Mnunrn WnlvnR MovED To ADopr”: read the image again and print the lines that hold ”MOVED” or ”PASSED”. Then Exhibit 3 of the December 2024 report, a picture of a table under a note offering ”a screen readable version” on request. Convert it to grayscale and read it with `config="--psm 6"`, the page segmentation mode for one uniform block of text. Parse each row with the chapter's `ROW` pattern, given below, keeping only the digits of each figure, since the table's dollars are whole and positive; print the lines with a year that don't parse. Then check each row: its twelve months should add up to its total. The chapter's ”When There Is No Text Layer: OCR” has the confidence scores too, and why they mislead.

```
# OCR page 7's one image of the 2018 minutes (minutes, from Step 2) with image_to_string(); print
 ↪  the image's size
#   and the lines that hold "MOVED" or "PASSED"
scan = minutes.pages[6].images[0].image
print(f"{scan.size} pixels")
for line in pytesseract.image_to_string(scan).splitlines():
```

```
    if "MOVED" in line or "PASSED" in line:
        print(line)
```

```
(2556, 3291) pixels

COUNCIL MEMBER WEAVER MOVED TO ADOPT ORDINANCE 8302
MORZEL SECONDED THE MOTION. THE MOTION PASSED 8:1 AT 9:08 P.M.
```

```
# exhibit_3 = page 15's one image of report_2024, .convert("L"); ocr_lines =
 ↪  image_to_string(exhibit_3, config="--psm 6"), split
#   into lines; print the image's size and the number of lines, then the first six words of lines
 ↪  4 to 6
exhibit_3 = PdfReader(report_2024).pages[14].images[0].image.convert("L")  # grayscale suits OCR
ocr_lines = pytesseract.image_to_string(exhibit_3, config="--psm 6").splitlines()
print(f"{exhibit_3.size} pixels, {len(ocr_lines)} lines of text")
for line in ocr_lines[4:7]:
    print(" ".join(line.split()[:6]))
```

```
(2545, 1331) pixels, 43 lines of text
2020 7,761,028 7,370,943 10,025,017 6,090,136 7,059,371
2021 8,059,343 7,608.759 10,351,245 8,666,637 9,229,065
2022 9,592,143 9,009,882 11,376,046 —11.242.614 10,608,781
```

```
# Given: the chapter's patterns. NUMBER is a figure as OCR may read it, with a stray dash, or
 ↪  periods for commas;
#   ROW is a line of the table: an optional category name, a year, then 13 figures (twelve months
 ↪  and the year's total)
NUMBER = r"[——-]?\d[\d.,]*"
ROW = re.compile(r"^(?P<label>[A-Za-z ]*?)\s*(?P<year>20\d\d)\s+(?P<figures>(?:" + NUMBER +
 ↪  r"\s+){13})")
```

```
# For each line that ROW matches, keep only the digits and spaces of match["figures"], and append
 ↪  [label, year] + the figures
#   to ocr_rows; for a line with a year that ROW doesn't match, print its first six words. Print
 ↪  how many rows parsed
ocr_rows = []
for line in ocr_lines:
    match = ROW.match(line)
    if match:
        figures = re.sub(r"[^\d\s]", "", match["figures"]).split()  # whole dollars: digits only
        ocr_rows.append([match["label"].strip(), match["year"]] + figures)
    elif re.search(r"\b20\d\d\b", line):
        print(f"set aside: {' '.join(line.split()[:6])}")
print(f"{len(ocr_rows)} of the table's 30 rows parsed")
```

```
set aside: ” 2022 489,417 405,726 646,263 552,449
29 of the table's 30 rows parsed
```

```
# ocr_df: a DataFrame of ocr_rows, its columns 0, 1, and 14 renamed category, year, and total;
 ↪  fill each category down its rows
#   Sum each row's twelve months (columns 2 to 13, as int) and print the largest gap between that
 ↪  sum and the total
ocr_df = pd.DataFrame(ocr_rows).rename(columns={0: "category", 1: "year", 14: "total"})
ocr_df["category"] = ocr_df["category"].replace("", pd.NA).ffill()  # each category's name starts
 ↪  only its first row
months = ocr_df.iloc[:, 2:14].astype(int).sum(axis=1)
gaps = (months - ocr_df["total"].astype(int)).abs()
print(f"largest gap between the months and the total: {gaps.max()} dollars")
```

```
largest gap between the months and the total: 2 dollars
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Change one parameter: the page segmentation mode.

Do: read `exhibit_3` again with `--psm 4` (one column of text of varying sizes) or `--psm 3` (Tesseract's default, which finds the layout itself), and count the lines that `ROW` matches. Time it with `time.time()`. **Share:** your mode, its number of lines and of rows parsed, and how many seconds it took.

```
# Microsprint: OCR exhibit_3 again with --psm 4 (or 3), count the lines ROW matches, and print
#   the number of lines, the rows parsed, and the seconds it took
for psm in ["4", "3"]:
    started = time.time()
    lines = pytesseract.image_to_string(exhibit_3, config=f"--psm {psm}").splitlines()
    parsed = 0
    for line in lines:
        if ROW.match(line):
            parsed += 1
    print(f"--psm {psm}: {len(lines)} lines, {parsed} rows parsed, {time.time() - started:.1f}
    ↪ seconds")
```

```
--psm 4: 45 lines, 29 rows parsed, 2.4 seconds
```

```
--psm 3: 60 lines, 30 rows parsed, 4.7 seconds
```

💬 Discuss

- Which mode parsed the most rows? Would you choose a mode by that count alone, or by what the sums say about the rows it parsed?
- Every parsed row adds up within $2. Name an OCR error that a row's sum can't catch.
- Chapter 9 calls a picture of a table the extreme case of soft enclosure. In week 3's terms, who is fenced out by it, and what does it cost them to get back in?

Step 8 — Check the extractions (6 minutes)

Extraction fails without errors, so check it, cheapest first: **count** the rows the page shows (nine), and check the **sums** (the five sources add up to the subtotal, and the subtotal and the two audit rows to the total, within a few dollars of rounding). Then **two documents agree**: OCR's 2024 total, from the picture of Exhibit 3, against the summary page's text. Last, **a sample coded by hand**: who was present at the December 5, 2024 roll call, measured as precision and recall against the nine members the page names. The chapter's "Checking an Extraction" lists eight checks, and gathers the first two into a function, `check_summary()`, that a pipeline can run on every file.

```
# Count: print len(summary_df) beside the page's 9. Sums, for both years at once, with figures =
↪ summary_df.set_index("source"):
#   print the first five rows' sum minus the subtotal, and the subtotal plus the two audit rows
↪ minus the total
print(f"{len(summary_df)} rows of figures; the page shows 9")
figures = summary_df.set_index("source")
subtotal = figures.loc["Total Sales & Use Tax based on current economic activity"]
audits = figures.loc["Sales Tax Audit Revenue"] + figures.loc["Use Tax Audit Revenue"]
print(f"sources minus subtotal: {(figures.iloc[:5].sum() - subtotal).to_dict()}")
print(f"subtotal plus audits minus total: {(subtotal + audits - figures.loc['Total Sales and Use
↪ Tax']).to_dict()}")
```

```
9 rows of figures; the page shows 9
sources minus subtotal: {'ytd_2023': 0, 'ytd_2024': 1}
subtotal plus audits minus total: {'ytd_2023': 0, 'ytd_2024': 0}
```

```
# Two documents: compare OCR's TOTAL row for 2024 (from ocr_df) with the summary page's total for
 ↪  2024 (from figures)
ocr_total = int(ocr_df[(ocr_df["category"] == "TOTAL") & (ocr_df["year"] ==
 ↪  "2024")]["total"].iloc[0])
text_total = figures.loc["Total Sales and Use Tax", "ytd_2024"]
print(f"OCR total {ocr_total:,} | text total {text_total:,} | match: {ocr_total == text_total}")
```

```
OCR total 173,908,431 | text total 173,908,431 | match: True
```

```
# A sample coded by hand: present_by_hand holds the nine members page 1 names. present_found: the
 ↪  capitalized words after
#   "present:" on its line of text (page 1, from Step 4); print precision, recall, and the
 ↪  members missed
present_by_hand = {"Adams", "Benjamin", "Brockett", "Folkerts", "Marquis", "Schuchard", "Speer",
 ↪  "Wallach", "Winer"}
match = re.search(r"present:(.*)$", text, re.MULTILINE)
present_found = set(re.findall(r"[A-Z][a-z]+", match.group(1)))
correct = present_found & present_by_hand
print(f"precision: {len(correct) / len(present_found):.2f} | recall: {len(correct) /
 ↪  len(present_by_hand):.2f}")
print(f"missed: {sorted(present_by_hand - present_found)}")
```

```
precision: 1.00 | recall: 0.56
missed: ['Adams', 'Speer', 'Wallach', 'Winer']
```

💬 Discuss

- Precision was perfect and recall was 0.56. Which of the two would a quick look at the output have shown you, and why?
- The sums say we copied the city's figures faithfully. Why don't they say the figures are right?
- The chapter's AI parser, Docling, wrote a complete-looking Exhibit 3 with two rows under the wrong label. Which of today's checks would you run on a parser's table first, and which would miss a wrong label?

Step 9 — Seven reports, one time series (11 minutes)

The chapter's case study: total sales and use tax from all seven year-end reports. The table's rules differ from year to year, but one row is all we need, so read it by position. First find each report's summary page by the sentence it begins with (”This report provides”), not by its number, since 2020's is on page 3. Then, for each report, `search()` finds the label ”Total Sales and Use Tax”, and a crop from the label's right end to the page's edge, as tall as its line, holds the row's figures; a `search()` with two groups reads the two years in the table's heading. Each row gives two years, the year before and this one: keep both from every report, compare the years printed twice, and plot each year's latest printing. The chapter's ”Case Study: Boulder Revenue Reports” has the figure, and the same steps as three functions.

```
# Given: the seven year-end reports, oldest first
REPORTS = ["revenue-report-2018.pdf", "revenue-report-2019.pdf", "revenue-report-2020.pdf",
           "revenue-report-2021-12.pdf", "revenue-report-2022-12.pdf",
           "revenue-report-2023-12.pdf", "revenue-report-2024-12.pdf"]
```

```
# For each report, find the page among its first four whose extract_text() holds "This report
 ↪  provides";
#   keep its page_number in the dictionary summary_pages, by the report's name; print
 ↪  summary_pages
```

```
summary_pages = {}
for name in REPORTS:
    with pdfplumber.open(get_saved_copy(name)) as pdf:
        for page in pdf.pages[:4]:
            if "This report provides" in page.extract_text():
                summary_pages[name] = page.page_number
print(summary_pages)
```

```
{'revenue-report-2018.pdf': 2, 'revenue-report-2019.pdf': 2, 'revenue-report-2020.pdf': 3,
 ↪  'revenue-report-2021-12.pdf': 2, 'revenue-report-2022-12.pdf': 2,
 ↪  'revenue-report-2023-12.pdf': 2, 'revenue-report-2024-12.pdf': 2}
```

```
# For each report's summary page: years = the groups of search(r"(20\d\d) +(20\d\d)"); found =
 ↪  search("Total Sales and Use Tax");
#   crop from found's x1 to the page's right edge, as tall as found, and split its text into
 ↪  figures
#   Append two rows to totals: the report, each year, and its figure (figures[0] for the year
 ↪  before, figures[1] for this one)
totals = []
for name in REPORTS:
    with pdfplumber.open(get_saved_copy(name)) as pdf:
        page = pdf.pages[summary_pages[name] - 1]  # page_number counts from 1
        years = page.search(r"(20\d\d) +(20\d\d)")[0]["groups"]
        found = page.search("Total Sales and Use Tax")[0]
        row = page.crop((found["x1"] + 1, found["top"] - 1, page.width, found["bottom"] + 1))
        figures = row.extract_text().split()
    totals.append({"report": name, "year": int(years[0]), "total": figures[0]})
    totals.append({"report": name, "year": int(years[1]), "total": figures[1]})
```

```
# Make totals a DataFrame, and its "total" column numbers, as in Step 6; for the years printed
 ↪  twice, print the count, min, max,
#   and how far apart the two printings are
totals = pd.DataFrame(totals)
totals["total"] = pd.to_numeric(totals["total"].str.replace(r"[$,]", "", regex=True))
printings = totals.groupby("year")["total"].agg(["count", "min", "max"])
twice = printings[printings["count"] == 2]
print(twice.assign(difference=twice["max"] - twice["min"]).to_string())
```

```
      count        min        max  difference
year
2018      2  140609751  140648710       38959
2019      2  141954650  141954650           0
2020      2  135195413  135195413           0
2021      2  152302653  152302653           0
2022      2  169112615  169112615           0
2023      2  175517129  175517129           0
```

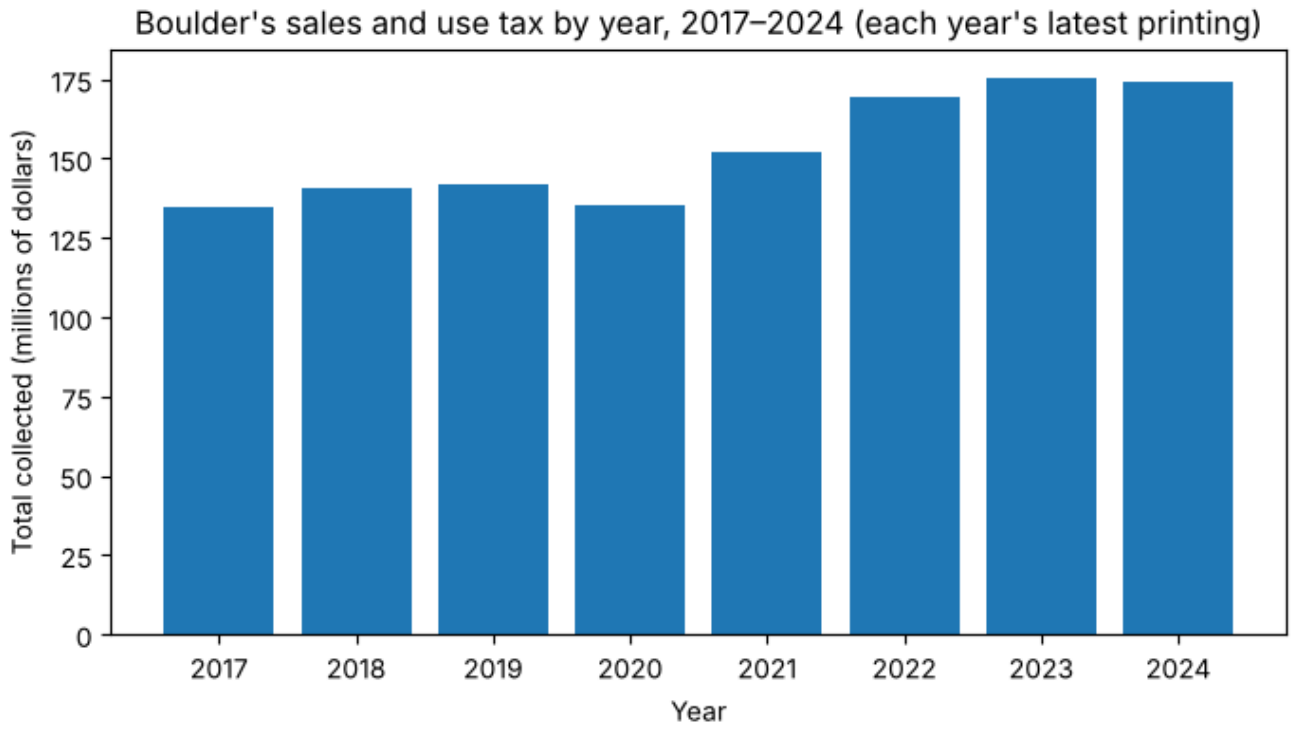
```
# latest: each year's last printing (groupby("year").last(), since REPORTS runs oldest to
 ↪  newest);
#   add change_pct, the percent change from the year before (pct_change() times 100, rounded to
 ↪  2), and print both columns
latest = totals.groupby("year").last()
latest["change_pct"] = (latest["total"].pct_change() * 100).round(2)
print(latest[["total", "change_pct"]].to_string())
```

```
          total  change_pct
year
```

```
2017  134538892         NaN
2018  140609751        4.51
2019  141954650        0.96
2020  135195413       -4.76
2021  152302653       12.65
2022  169112615       11.04
2023  175517129        3.79
2024  173908431       -0.92
```

```python
# Import matplotlib.pyplot as plt; plot latest's totals in millions of dollars as bars, with a
↪  title and labeled axes
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(latest.index, latest["total"] / 1e6)
ax.set_title("Boulder's sales and use tax by year, 2017–2024 (each year's latest printing)")
ax.set_xlabel("Year")
ax.set_ylabel("Total collected (millions of dollars)")
plt.show()
```



💬 Discuss

- Sales and use tax fell 4.76% in 2020 and rose 12.65% in 2021. What would you need before calling that a pandemic effect?
- The 2018 total is printed two ways, $38,959 apart. Which would you use, and what would you write in the data's notes?
- Week 3 named oversight as a value for public-interest data work. What can someone outside City Hall check with this series that they couldn't with the PDFs alone?

Step 10 — What did we learn? (4 minutes)

In a few sentences, as comments or in a Markdown cell:

- Which pages we opened today needed OCR, and how could you tell without looking at them?
- Name one extraction that failed without an error today, and the check that caught it.
- How did Boulder's sales and use tax change from 2019 to 2024, and what lets you trust the figures enough to put eight years side by side?
- For a hundred council minutes, would you start from pypdf with repairs or from pdfplumber? What would you check first?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, collect three PDFs from one government source, politely, into `pdfs/`: Step 0's download (User-Agent, save the bytes) and Step 2's `get_saved_copy()` (download once, pause after each), and Step 3's `%PDF-` check.
- Step 2, open one and print its page count and page 1's raw text: Steps 1, 2, and 4.
- Step 3, write a cleanup function and show a passage before and after: Step 5's four repairs, and its microsprint's check on a second file.
- Step 4, extract one element with `re.findall()`: Step 5's five patterns.
- Step 5, loop the pipeline over all three PDFs into a DataFrame: Step 9's loop over seven reports, and Step 2's characters per page.
- Step 6, plot your element's count per document and look at characters per page: Step 9's plot, and Step 2's routing.
- Step 7, interpret, and judge extraction quality: Step 8's checks.

```
# Your answers here, as comments
# OCR: six of the nine pages of the December 19, 2000 minutes (pages 2-4 and 7-9) have an image
 ↪  and no text, and Exhibit 3
#   in the December 2024 report is one image with 265 characters beside it. Step 2's counts said
 ↪  so: an image and
#   next to no text. The 2018 minutes have text on every page, but it is the scanner's, and
 ↪  Tesseract read the motion better.
# Silent failures: pdfplumber's default table stopped at "Use Tax Audit Revenue": search() put
 ↪  the table's bottom at 375
#   points and the totals row at 380 to 391, and in 2018 at 523 and 529 to 540. Of the join
 ↪  repair's 30 joins on the
#   2023-12-07 minutes, 3 looked suspicious and 1 was wrong: "A-FWallach". The roll-call pattern
 ↪  had precision 1.00 but
#   recall 0.56, missing 4 of 9 members. Tesseract's --psm 6 set 1 of Exhibit 3's 30 rows aside.
# Revenue: 141,954,650 in 2019, down 4.76% to 135,195,413 in 2020, up 12.65% and 11.04% in 2021
 ↪  and 2022, then 3.79%,
#   and down 0.92% to 173,908,431 in 2024. Five of the six years printed twice agree to the
 ↪  dollar, and 2018 differs by
#   38,959, so we used the latest printing. The 2024 summary has its 9 rows and adds up within
 ↪  $1, and OCR's 2024 total
#   matches the text's. Dollars of each year, not adjusted for inflation.
# pdfplumber's text needed fewer repairs on these minutes, but the repairs are checkable either
 ↪  way: start with a hand-coded
#   sample of roll calls, and count joins and misses on a second year's files before trusting a
 ↪  hundred.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter should know? Some places to look:

- **Outputs that changed.** In Step 4, pypdf's layout mode kept ”Council Chambers” and ”Carried” whole, where the chapter prints ”C ham bers” and ”Car r ied” and builds a paragraph on it (”Plain mode breaks lines, and layout mode breaks words”). What did yours print? With Tesseract 5.3.4, Step 7 parsed 29 of Exhibit 3's rows and set 1 aside; the chapter, with 5.5.3, reports 28 and 2. Did your copy match either?

- **A microsprint that would make a good exercise.** Step 6's microsprint asked whether the default table finder drops the totals row in other years too. It does in all seven reports, and ”Finding Tables” shows it only for 2024. Is that worth a sentence there, or an Additional Exercise?
- **Code the lab wrote differently.** The lab wrote the chapter's `page_profile()`, `clean_page_text()`, `summary_frame()`, `check_summary()`, and the case study's three functions as straight lines. Which read more easily that way, and where did you miss the function?
- **A sentence that confused you.** Did any of the chapter's explanations of pypdf's two modes, the join rule, or the table settings leave you guessing during the lab?
- **A check the chapter could add.** Step 5 showed the tally pattern finds votes only in the motion tables. Is there a short check that would count what it misses?

`handouts/common/revision-framework.md` says how to propose a revision.